

Code & Output for CN Lab

Assignment 7: Error Detection using CRC.

CRC Error Detection Program (following the algorithm)

```
def xor(a, b):
```

```
    # XOR operation: compare bits, return '0' if same, '1' if different
```

```
    result = ""
```

```
    for i in range(1, len(b)):
```

```
        result += '0' if a[i] == b[i] else '1'
```

```
    return result
```

```
def mod2div(dividend, divisor):
```

```
    # Step 2: Divide B(x) by G(x) using Modulo-2 Division
```

```
    pick = len(divisor)
```

```
    tmp = dividend[0:pick]
```

```
    while pick < len(dividend):
```

```
        if tmp[0] == '1':
```

```
            tmp = xor(divisor, tmp) + dividend[pick]
```

```
        else:
```

```
            tmp = xor('0'*pick, tmp) + dividend[pick]
```

```
        pick += 1
```

```
    # Final step for last bits
```

```
    if tmp[0] == '1':
```

```
        tmp = xor(divisor, tmp)
```

```
    else:
```

```
        tmp = xor('0'*pick, tmp)
```

```
    return tmp # remainder R(x)
```

```
def encode(data, key):
```

```
    # Step 1: Append zeros (degree of generator polynomial)
```

```
    appended = data + '0'*(len(key)-1)
```

```
    # Step 2: Divide and get remainder R(x)
```

```
    remainder = mod2div(appended, key)
```

```
    # Step 3: Define  $T(x) = B(x) - R(x) \rightarrow$  codeword
```

```
    return data + remainder
```

```
# ----- MAIN PROGRAM -----  
  
data = input("Enter the Data Bits: ")  
#data = "100100" # Example bit string (B)  
key = "1101" # Generator polynomial (G)  
print("Original Data (B):", data)  
codeword = encode(data, key)  
print("Transmitted Codeword (T):", codeword)  
# Step 5: Receiver checks  
  
received = codeword # assume no error  
remainder = mod2div(received, key)  
if int(remainder) == 0:  
    print("No error detected (T = T').")  
else:  
    print("Error detected! Retransmission required.")
```

Output:

```
Enter the Data Bits: 100100  
Original Data (B): 100100  
Transmitted Codeword (T): 100100001  
No error detected (T = T').
```

Assignment 8: Frame Sorting Technique

#Frame Sorting Technique

Frames arriving out of order

```
frames = [  
    (2, "or"), # Frame 2  
    (1, "W"),  # Frame 1  
    (3, "ld")  # Frame 3  
]  
  
print("Received Frames:", frames)  
  
# Sort by sequence number  
frames.sort(key=lambda x: x[0])  
print("Sorted Frames:", frames)  
  
# Reconstruct message  
message = ""  
for number, data in frames:  
    message += data  
  
print("Reconstructed Message:", message)
```

Output:

Received Frames: [(2, 'or'), (1, 'W'), (3, 'ld')]

Sorted Frames: [(1, 'W'), (2, 'or'), (3, 'ld')]

Reconstructed Message: World.

Assignment 9: Hamming Code

Interactive Hamming Code Program

```
def calculate_parity_bits(data_length):  
    r = 0  
    while (2**r < data_length + r + 1):  
        r += 1  
    return r  
  
def generate_hamming_code(data):  
    r = calculate_parity_bits(len(data))  
    hamming = ['0'] * (len(data) + r)  
  
    # Place data bits in non-parity positions  
    j = 0  
    for i in range(1, len(hamming) + 1):  
        if (i & (i - 1)) != 0: # not a power of 2 → data position  
            hamming[i - 1] = data[j]  
            j += 1  
  
    # Calculate parity bits  
    for i in range(r):  
        parity_pos = 2**i  
        parity = 0  
        for j in range(1, len(hamming) + 1):  
            if j & parity_pos:  
                parity ^= int(hamming[j - 1])  
        hamming[parity_pos - 1] = str(parity)  
  
    return ''.join(hamming)  
  
def detect_error(hamming_code):  
    r = calculate_parity_bits(len(hamming_code))  
    error_pos = 0  
    for i in range(r):  
        parity_pos = 2**i  
        parity = 0  
        for j in range(1, len(hamming_code) + 1):
```

```

        if j & parity_pos:
            parity ^= int(hamming_code[j - 1])
        if parity != 0:
            error_pos += parity_pos
    return error_pos

# ----- MAIN PROGRAM -----

print("Hamming Code Program")

choice = input("Type 'generate' to create Hamming code OR 'check' to detect error: ").lower()
data = input("Enter data bits : ")
if choice == "generate":
    hamming_code = generate_hamming_code(data)
    print("Generated Hamming Code:", hamming_code)

elif choice == "check":
    hamming_code = data # here user enters the received hamming code
    error_position = detect_error(hamming_code)
    if error_position == 0:
        print("No error detected.")
    else:
        print("Error detected at position:", error_position)

else:
    print("Invalid choice. Please type 'generate' or 'check'.")

```

OUTPUT

Hamming Code Program

Type 'generate' to create Hamming code OR 'check' to detect error: generate

Enter data bits : 1011

Generated Hamming Code: 1011010

Hamming Code Program

Type 'generate' to create Hamming code OR 'check' to detect error: check

Enter data bits : 1011010

No error detected.

Assignment 10 : FIFO IPC

Server Code:

```
import time

import os

SERVER_FILE = "server_request.txt"
CLIENT_FILE = "client_reply.txt"

print("Server is running... waiting for client request.")

while True:

    # Wait until client writes a request
    while not os.path.exists(SERVER_FILE):
        time.sleep(1)

    with open(SERVER_FILE, "r") as f:
        filename = f.read().strip()

    if filename == "exit":
        print("Server shutting down.")
        break

    print(f"Client requested file: {filename}")

    try:
        with open(filename, "r") as f:
            file_contents = f.read()
    except FileNotFoundError:
        file_contents = "Error: File not found."

    # Write reply for client
    with open(CLIENT_FILE, "w") as f:
        f.write(file_contents)

    # Remove request file so next request can come
    os.remove(SERVER_FILE)

    print("Reply sent to client.")
```

Client Code:

```
import time

import os

SERVER_FILE = "server_request.txt"

CLIENT_FILE = "client_reply.txt"

filename = input("Enter the filename you want to read: ")

# Send request

with open(SERVER_FILE, "w") as f:

    f.write(filename)

# Wait for reply

while not os.path.exists(CLIENT_FILE):

    time.sleep(1)

with open(CLIENT_FILE, "r") as f:

    response = f.read()

print("\n--- File Contents ---")

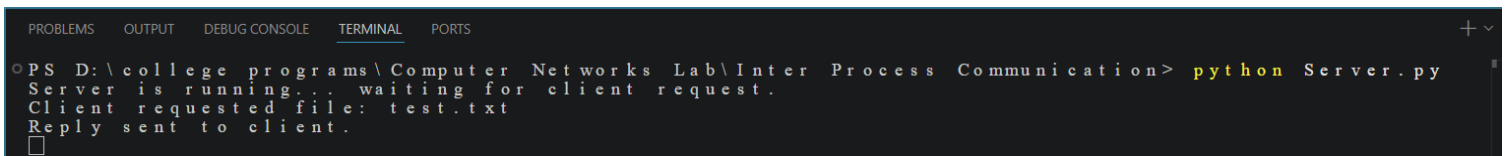
print(response)

print("-----")

# Clean up

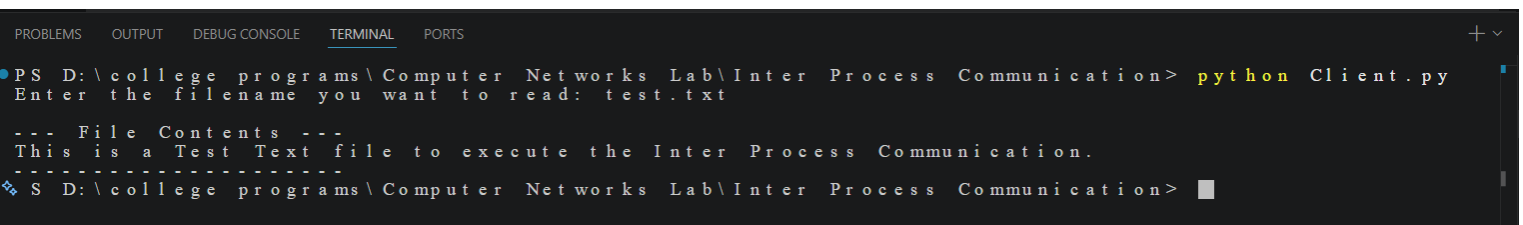
os.remove(CLIENT_FILE)
```

#SERVER OUPUT

A screenshot of a terminal window with a dark background. The terminal has tabs at the top: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The output shows the execution of 'python Server.py'. The text displayed is: 'PS D:\college programs\Computer Networks Lab\Inter Process Communication> python Server.py', 'Server is running... waiting for client request.', 'Client requested file: test.txt', 'Reply sent to client.', and a cursor on a new line.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\college programs\Computer Networks Lab\Inter Process Communication> python Server.py
Server is running... waiting for client request.
Client requested file: test.txt
Reply sent to client.
█
```

#Client Output

A screenshot of a terminal window with a dark background. The terminal has tabs at the top: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The output shows the execution of 'python Client.py'. The text displayed is: 'PS D:\college programs\Computer Networks Lab\Inter Process Communication> python Client.py', 'Enter the filename you want to read: test.txt', '--- File Contents ---', 'This is a Test Text file to execute the Inter Process Communication.', and a cursor on a new line.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\college programs\Computer Networks Lab\Inter Process Communication> python Client.py
Enter the filename you want to read: test.txt
--- File Contents ---
This is a Test Text file to execute the Inter Process Communication.
-----
PS D:\college programs\Computer Networks Lab\Inter Process Communication> █
```

Assignment 11: TCP Socket Programming

Server Code:

```
import socket

# Create socket

server = socket.socket()

server.bind(('localhost', 12345)) # Bind to localhost and port 12345

server.listen(1)          # Listen for 1 connection


print("Server is waiting for connection...")

conn, addr = server.accept()

print("Connected to:", addr)


# Receive filename

filename = conn.recv(1024).decode()

try:

    with open(filename, 'r') as f:

        data = f.read()

        conn.send(data.encode())    # Send file contents

except FileNotFoundError:

    conn.send(b"File not found!") # Send error message


conn.close()

server.close()
```

SERVER OUPUT:

```
PS D:\college programs\Computer Networks Lab\TCP Socket> python server.py
Server is waiting for connection...
Connected to: ('127.0.0.1', 53589)
PS D:\college programs\Computer Networks Lab\TCP Socket>
```


Client Code:

```
import socket

# Create socket
client = socket.socket()

client.connect(('localhost', 12345)) # Connect to server

# Ask user for filename
filename = input("Enter filename: ")
client.send(filename.encode())

# Receive and print file contents
data = client.recv(4096).decode()
print("From Server:\n", data)

client.close()
```

#Client Output:

```
PS D:\college programs\Computer Networks Lab\TCP Socket> python Client.py
Enter filename: test.txt
From Server:
  This is a test file to execute the TCP Socket Programming.
PS D:\college programs\Computer Networks Lab\TCP Socket>
```

Assignment 12 : Chat Application using TCP

Server Code:

```
import socket

# Create a socket
server_socket = socket.socket()

server_socket.bind(('localhost', 12345)) # Bind to localhost and port 12345
server_socket.listen(1) # Listen for 1 connection

print("Server is waiting for connection...")

conn, addr = server_socket.accept()
print("Connected with:", addr)

while True:
    data = conn.recv(1024).decode()
    if data.lower() == 'bye':
        print("Client ended chat.")
        break
    print("Client:", data)
    msg = input("Server: ")
    conn.send(msg.encode())
    if msg.lower() == 'bye':
        break

conn.close()
server_socket.close()
```

Client Code:

```
import socket

# Create a socket
client_socket = socket.socket()

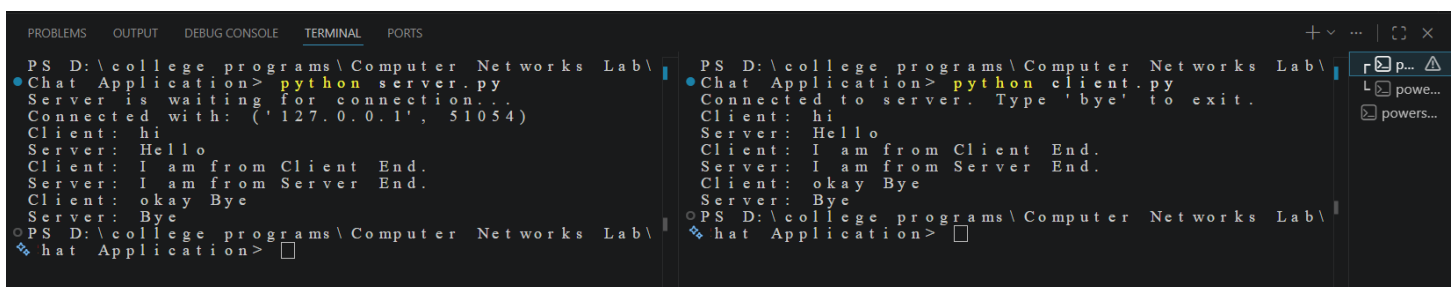
client_socket.connect(('localhost', 12345)) # Connect to server

print("Connected to server. Type 'bye' to exit.")

while True:
    msg = input("Client: ")
    client_socket.send(msg.encode())
    if msg.lower() == 'bye':
        break
    data = client_socket.recv(1024).decode()
    print("Server:", data)
    if data.lower() == 'bye':
        break

client_socket.close()
```

#OUPUT (Server + Client)



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\college programs\Computer Networks Lab\
• Chat Application> python server.py
Server is waiting for connection...
Connected with: ('127.0.0.1', 51054)
Client: hi
Server: Hello
Client: I am from Client End.
Server: I am from Server End.
Client: okay Bye
Server: Bye
PS D:\college programs\Computer Networks Lab\
• hat Application>

PS D:\college programs\Computer Networks Lab\
• Chat Application> python client.py
Connected to server. Type 'bye' to exit.
Client: hi
Server: Hello
Client: I am from Client End.
Server: I am from Server End.
Client: okay Bye
Server: Bye
PS D:\college programs\Computer Networks Lab\
• hat Application>
```